

Atencion

Esto es un simple copypaste de www.swi-prolog.org
(recorte cosas que no me parecieron necesarias de las descripciones.
Exercise caution in use.)

=(?Term1, ?Term2)

Unificador, no respeta occurs check

sort(+List, -Sorted)

True if Sorted can be unified with a list holding the elements of List, sorted to the standard order of terms. Elimina duplicados.

msort(+List, -Sorted)

Igual a sort pero sin eliminar duplicados.

length(?List, ?Length)

True if Length represents the number of elements in List. This predicate is a true relation and can be used to find the length of a list or produce a list (holding variables) of length Length.

nth1(?Index, ?List, ?Elem)

Is true when Elem is the Index'th element of List. Counting starts at 1.

nth0(?Index, ?List, ?Elem)

True when Elem is the Index'th element of List. Counting starts at 0.

nth0(?N, ?List, ?Elem, ?Rest)

Select/insert element at index. True when Elem is the N'th (0-based) element of List and Rest is the remainder (as in `select/3`) of List. For example:

```
?- nth0(I, [a,b,c], E, R).  
I = 0, E = a, R = [b, c] ;
```

member(?Elem, ?List)

True if Elem is a member of List.

append(?List1, ?List2, ?List1AndList2)

Concatena List1 y List2

append(+ListOfLists, ?List)

Concatenate a list of lists. Is true if ListOfLists is a list of lists, and List is the concatenation of these lists.

last(?List, ?Last)

Succeeds when Last is the last element of List. This predicate is semidet if List is a list and multi if List is a partial list. (use `append(_, [Last], List)` as a portable alternative)

between(+Low, +High, ?Value)

Low and High are integers, $High \geq Low$. If Value is an integer, $Low \leq Value \leq High$. When Value is a variable it is successively bound to all integers between Low and High.

is_list(+Term)

True if Term is bound to the empty list `[]` or a compound term with name `'[]'` and arity 2 and the second argument is a list.

list_to_set(+List, ?Set)

True when Set has the same elements as List in the same order. The left-most copy of duplicate elements is retained. List may contain variables. Elements E1 and E2 are considered duplicates iff $E1 == E2$ holds.

union(+Set1, +Set2, -Set3)

True if Set3 unifies with the union of the lists Set1 and Set2.

intersection(+Set1, +Set2, -Set3)

True if Set3 unifies with the intersection of Set1 and Set2.

subset(+SubSet, +Set)

True if all elements of SubSet belong to Set as well. A set is defined to be an unordered list without duplicates. Elements are considered duplicates if they can be unified.

subtract(+Set, +Delete, -Result)

Delete all elements in Delete from Set. (Mismo reminder que subset)

select(?Elem, ?List1, ?List2)

Is true when List1, with Elem removed, results in List2. This implementation is deterministic if the last element of List1 has been selected.

select(?X, ?XList, ?Y, ?YList)

Select from two lists at the same position. True if XList is unifiable with YList apart a single element at the same position that is unified with X in XList and with Y in YList. A typical use for this predicate is to replace an element, as shown in the example below. All possible substitutions are performed on backtracking.

```
?- select(b, [a,b,c,b], 2, X).  
X = [a, 2, c, b] ;  
X = [a, b, c, 2] ;  
false.
```

delete(+List1, @Elem, -List2)

Delete matching elements from a list. True when List2 is a list with all elements from List1 except for those that unify with Elem. Matching Elem with elements of List1 is uses \+ Elem \= H, which implies that Elem is not changed.

reverse(?List1, ?List2)

Is true when the elements of List2 are in reverse order compared to List1. This predicate is deterministic if either list is a proper list. If both lists are partial lists backtracking generates increasingly long lists.

atom(@Term)

True if Term is bound to an atom.

number(@Term)

True if Term is bound to a rational number (including integers) or a floating point number.

numlist(+Low, +High, -List)

List is a list [Low, Low+1, ... High]. Fails if High < Low.

sum_list(+List, -Sum)

Sum is the result of adding all numbers in List.

flatten(+NestedList, -FlatList)

Is true if FlatList is a non-nested version of NestedList. Note that empty lists are removed.

Extra logica

-Number is +Expr

True when Number is the value to which Expr evaluates. Typically, `is/2` should be used with unbound left operand. If equality is to be tested, `:=/2` should be used. For example:
`?- 1 is sin(pi/2).` Fails! `sin(pi/2)` evaluates to the float 1.0, which does not unify with the integer 1.
`?- 1 := sin(pi/2).` Succeeds as expected.

@Term1 \= @Term2

Unify X and Y and if this succeeds, fail (rolling the unification back)

@Term1 == @Term2

True if Term1 is equivalent to Term2. A variable is only identical to a sharing variable.

+Expr1 ::= +Expr2

True if expression Expr1 evaluates to a number equal to Expr2.

+Expr1 =\= +Expr2

True if expression Expr1 evaluates to a number non-equal to Expr2.

+Expr1 > +Expr2

True if expression Expr1 evaluates to a larger number than Expr2.
`+Expr1 < +Expr2`
`+Expr1 <= +Expr2`
`+Expr1 >= +Expr2`

abs(+Expr)

Evaluate Expr and return the absolute value of it.

abs(+Number, -Absolute)

Unify 'Absolute' with the absolute value of 'Number'.

max(+Expr1, +Expr2)

Evaluate to the larger of Expr1 and Expr2. Both arguments are compared after converting to the same type, but the return value is in the original type. For example, `max(2.5, 3)` compares the two values after converting to float, but returns the integer 3. If both values are numerical equal the returned max is of the type used for the comparison.

min(+Expr1, +Expr2)

+IntExpr1 mod +IntExpr2

Modulo, defined as $\text{Result} = \text{IntExpr1} - (\text{IntExpr1} \text{ div } \text{IntExpr2}) \times \text{IntExpr2}$, where div is floored division.

+IntExpr1 gcd +IntExpr2

Result is the greatest common divisor of IntExpr1 and IntExpr2. The GCD is always a positive integer. If either expression evaluates to zero the GCD is the result of the other expression.

var(@Term)

True if Term currently is a free variable.

nonvar(@Term)

ground(@Term)

True if Term holds no free variables.

disclaimer

trace, notrace, make y halt no los
agrego porque ¿? pero existen y estan
listados